

Viva Q&A - Assignment 2
Path Planning & Trajectory Generation - RMPC

Contents

1. GENERAL / ARCHITECTURE QUESTIONS	2
2. A* ALGORITHM QUESTIONS	4
3. RRT QUESTIONS	7
4. PRM QUESTIONS	9
5. TRAJECTORY GENERATION QUESTIONS	11
6. UTILITIES / PRE-BUILT CODE QUESTIONS	15
7. COMPARISON / CONCEPTUAL QUESTIONS	16
8. EDGE CASES / TRICKY QUESTIONS	18
9. THEORY / DEEPER UNDERSTANDING	20

1. GENERAL / ARCHITECTURE QUESTIONS

Q1: Walk me through what happens when you run main.py, step by step.

A:

1. A `LatticeGraph` is created. `initialise_graph(10, 10, 10)` generates 400 vertices (10 rows $\times$ 10 cols $\times$ 4 orientations: 0, 90, 180, 270 degrees) and connects them with straight edges (weight 1) and arc edges (weight π).
2. Two `ObstaclesGrid` objects are created: one at full resolution (100 $\times$ 100) and one scaled down for plotting.
3. Five rectangular obstacle regions are marked as `True` in the boolean grid.
4. `update_obstacles()` iterates every edge, samples points along it (line or arc), and sets the weight to ∞ for any edge that crosses an obstacle.
5. A* is called with start (1,8,90) and goal (8,2,270) on the lattice graph. It returns a list of (row, col, angle) tuples.
6. RRT is created with the same start/goal in 2D and `plan()` builds a tree incrementally until the goal is reached.
7. PRM is created, `construct_roadmap()` samples 200 free points and connects k=10 nearest neighbors, then `plan()` runs Dijkstra on the roadmap.
8. If A* found a path, `path_interpolation()` densifies it (sampling along straight segments and arcs), and `resample_path()` generates a full trajectory with velocity/acceleration profiles.
9. Results are plotted with matplotlib.

Q2: Why do you use three different planners? What does each one demonstrate?

A: Each planner represents a different paradigm:

- **A* on Lattice:** Graph-based, deterministic, optimal. Demonstrates structured search with orientation constraints. Good when the environment can be discretized.
- **RRT:** Sampling-based, single-query, probabilistically complete but not optimal. Demonstrates rapid exploration of continuous spaces. Good for high-dimensional spaces or kinematic constraints.
- **PRM:** Sampling-based, multi-query, two-phase. Demonstrates the idea of building a reusable roadmap. Good when you need to solve many queries in a static environment.

Together they show the trade-offs between optimality, speed, completeness, and reusability in path planning.

Q3: What is the difference between a path and a trajectory?

A: A **path** is a purely geometric object: a sequence of positions (and optionally orientations) in space with no notion of time. A **trajectory** is a path parameterized by time: at each instant t , the robot has a specific position (x, y) , orientation θ , velocity v , acceleration a , and angular velocity ω . The trajectory tells the robot not just *where* to go but *when* to be there and *how fast* to move.

Q4: Why does the lattice have 4 orientations? What would happen with 8?

A: Four orientations (0, 90, 180, 270) correspond to the 4 cardinal directions (right, up, left, down). This captures the robot's heading at each node. With 8 orientations (adding 45, 135, 225, 315), the lattice would have 800 vertices instead of 400, more edges (including 45-degree arcs), and finer angular resolution. This would produce smoother paths with more diverse headings, but at the cost of increased computational time and memory. Four orientations are a reasonable trade-off for this assignment.

Q5: What does the `lattice_cell_size = 10` represent?

A: Each lattice cell spans 10 units in the obstacle map. So a 10×10 lattice grid maps to a 100×100 obstacle map. When checking if an edge is valid, points are sampled along the edge in map coordinates (multiplied by cell size). This scaling lets the obstacle map have finer resolution than the planning graph.

Q6: What is the `scaler = 3.0` in `main.py` used for?

A: The scaler is used to create a lower-resolution version of the obstacle map for plotting purposes (`obs_plot`). The plot grid is $(100/3) \times (100/3) \approx 33 \times 33$. This is purely for visualization; the actual planning uses the full 100×100 grid.

2. A* ALGORITHM QUESTIONS

Q7: Explain A* in your own words. How does it work?

A: A* is a best-first graph search algorithm. It maintains an open set (priority queue) of nodes to explore, ordered by $f(n) = g(n) + h(n)$, where $g(n)$ is the actual cost from start to n , and $h(n)$ is a heuristic estimate from n to the goal. At each step, it pops the node with the lowest f , expands its neighbors, and updates costs if a better path is found. It stops when the goal is popped. With an admissible heuristic, it guarantees the optimal path.

Q8: What is the difference between A* and Dijkstra?

A: Dijkstra is a special case of A* where $h(n) = 0$ for all nodes. It expands nodes purely by g -cost (distance from start), exploring equally in all directions. A* uses the heuristic $h(n)$ to guide the search toward the goal, so it typically expands fewer nodes and is faster. Both guarantee optimal paths (A* requires an admissible heuristic).

Q9: What does "admissible" mean? Is your heuristic admissible?

A: A heuristic is admissible if it **never overestimates** the true cost from any node to the goal: $h(n) \leq h^*(n)$ for all n , where $h^*(n)$ is the true optimal cost. My heuristic is Euclidean distance: $h(n) = \sqrt{(r_n - r_g)^2 + (c_n - c_g)^2}$. Since the straight-line distance is always $\leq$ any actual path (which must follow edges of weight ≥ 1), it never overestimates. Therefore it is admissible.

Q10: What does "consistent" (monotone) mean? Is your heuristic consistent?

A: A heuristic is consistent if for every node n and every neighbor n' : $h(n) \leq \text{cost}(n, n') + h(n')$. This is the triangle inequality applied to the heuristic. Euclidean distance satisfies the triangle inequality, so yes, my heuristic is consistent. Consistency implies admissibility and guarantees that A* never needs to reopen (re-expand) a node from the closed set.

Q11: What happens if the heuristic is NOT admissible?

A: A* loses its optimality guarantee. It may return a suboptimal path because it could prematurely expand the goal through a path that appears cheaper (due to the overestimating heuristic) but is actually more expensive. It would still find a path if one exists, but not necessarily the shortest one. This is sometimes acceptable (weighted A*) when speed is more important than optimality.

Q12: Why do you check the goal AFTER popping, not when adding to the open set?

A: If I check when adding, I might discover the goal through a suboptimal path first. Another path might reach the goal with lower g -cost but hasn't been discovered yet. By checking after popping, the priority queue guarantees that the first time the goal is popped, it has the minimum f -cost. With an admissible and consistent heuristic, this is also the minimum g -cost, guaranteeing optimality.

Q13: Explain the lazy deletion pattern. Why is it needed?

A: Python's `PriorityQueue` does not support a **decrease-key** operation. When I find a better path to a node, I cannot update its priority in-place. Instead, I push a new entry with the updated priority. This means the queue may contain multiple entries for the same node. When I pop a node that is already in the closed set, I skip it ("lazy deletion"). This is correct because the first time a node is popped, it has the best f -cost; subsequent pops of the same node are stale.

Q14: What is the time complexity of your A* implementation?

A: $O(V \log V + E \log V)$ where V = vertices (400) and E = edges. Each vertex is popped from the priority queue at most once ($O(V \log V)$). For each vertex, we examine its neighbors (total E edges across all vertices). Each edge may cause a push to the queue ($O(\log V)$ per push). With lazy deletion, up to E entries may be pushed, so it is $O(E \log E) = O(E \log V)$ since $E \leq V^2$.

Q15: Could you use a different heuristic? What are the alternatives?

A:

- **Manhattan distance** ($|dr| + |dc|$): Admissible but less informed. Would overcount for diagonal paths.
- **Chebyshev distance** ($\max(|dr|, |dc|)$): Admissible, less tight than Euclidean.
- **Zero heuristic** ($h = 0$): Admissible. Degenerates A* into Dijkstra. Explores more nodes.
- **Weighted Euclidean** ($w \cdot \sqrt{dr^2 + dc^2}$ with $w > 1$): Not admissible. Faster but may be suboptimal (Weighted A*).

Euclidean is the tightest standard admissible heuristic for this grid, meaning fewer node expansions.

Q16: What happens if there is no valid path from start to goal?

A: The open set eventually empties (all reachable nodes have been expanded without finding the goal). The while loop exits, we print "no path found" and return `None`. In `main.py`, the check `if path:` prevents the trajectory generation from running on a `None` path.

Q17: Why does your heuristic ignore the angle component of the vertex?

A: The heuristic only needs to be a lower bound on the true cost. Since the Euclidean distance in (row, col) already underestimates the cost (because any path is at least as long as the straight line), adding angle considerations is unnecessary for admissibility. Including angles could make the heuristic more informed (tighter), but the added complexity is not worth it for this problem size.

Q18: How does the path reconstruction work?

A: During search, each time I update a node's best path, I record its parent: `parent_node[n] = curr`. After finding the goal, I start from the goal and follow parent pointers back to start, building the path in reverse. Then I reverse the list. This is standard backtracking, used in all graph search algorithms.

Q19: What does `cal_expand_cost` do?

A: It returns the edge weight from the edge dictionary: `edge_dict[(v1, v2)]`. For straight edges this is 1, for arcs this is π , and for blocked edges (through obstacles) this is ∞ . A* will never choose an infinite-cost edge as part of an optimal path.

Q20: What is the open set and what is the closed set?

A: The **open set** contains nodes that have been discovered but not yet expanded (their neighbors haven't been examined). It is implemented as a priority queue ordered by f -cost. The **closed set** contains nodes that have already been expanded. Once a node is in the closed set (with a consistent heuristic), its shortest path has been found and it won't be reconsidered.

3. RRT QUESTIONS

Q21: Explain the RRT algorithm in your own words.

A: RRT incrementally builds a tree rooted at the start. At each iteration: (1) sample a random point in the map, (2) find the nearest node in the tree to that sample, (3) steer from the nearest node toward the sample by a fixed step size, creating a new node, (4) check if the path from nearest to new node is collision-free, (5) if so, add the new node to the tree, (6) check if the new node is close enough to the goal. The tree grows by exploring the space randomly until it reaches the goal.

Q22: Is RRT optimal? Is it complete?

A: RRT is **not optimal** — it finds *a* path, not the shortest one. The path depends on the random samples and tree structure. RRT is **probabilistically complete**: as iterations $\rightarrow \infty$, the probability of finding a path (if one exists) $\rightarrow 1$. It is not deterministically complete because any finite run might fail.

Q23: What is the difference between RRT and RRT*?

A: RRT* adds two operations: (1) **near-neighbor search**: when adding a new node, check all nodes within a radius (not just the nearest) to find the lowest-cost parent, (2) **rewiring**: after adding the new node, check if any nearby nodes would have a shorter path through the new node, and rewire the tree accordingly. RRT* is **asymptotically optimal**: as iterations $\rightarrow \infty$, the path cost converges to the optimal cost. Standard RRT does not converge to the optimum.

Q24: How does the steer function work?

A: It computes the direction vector from the nearest node to the random sample: (dx, dy) . If the sample is within **step_size**, it returns the sample directly. Otherwise, it normalizes the direction to a unit vector $(dx/d, dy/d)$ and scales by **step_size** to get the new position. This ensures the tree grows by at most **step_size** per iteration, enabling reliable collision checking.

Q25: How does your collision checking work in RRT?

A: I sample points along the line segment between the nearest node and the new node. The number of samples equals $\max(\text{int}(\text{distance}), 1)$, giving roughly one sample per unit distance. At each sample, I interpolate the position using parameter $t \in [0, 1]$, convert to integer grid coordinates, and check: (1) if it's within the map bounds, (2) if the obstacle grid at that position is True. If any sample fails, the edge is in collision.

Q26: Why do you use step_size as the goal threshold?

A: Since RRT operates in continuous space with random sampling, the probability of landing exactly on the goal coordinates is essentially zero (floating-point). Using **step_size** as a proximity threshold means “if I’m within one step of the goal, I can reach it.” This is standard practice in sampling-based planners.

Q27: How does `step_size` affect RRT's behavior?**A:**

- **Too small:** The tree grows slowly, requiring many iterations to traverse the map. May not reach the goal within `max_iter`.
- **Too large:** The tree might jump over narrow passages between obstacles. Collision checks become less reliable because a long segment might pass through a thin obstacle without any sample landing on it.
- **`step_size = 5`** on a 100×100 map is a good balance: the tree covers the map in ~ 20 steps, and the collision check samples 5 points per edge, adequate for the obstacle sizes.

Q28: Why is the RRT path typically suboptimal (jagged)?

A: The tree structure is dictated by random samples. When a random point is sampled, the tree extends toward it from the nearest existing node, regardless of whether that is the best direction. The resulting path follows the tree's branching structure, which is generally not the shortest route. Path smoothing (e.g., shortcutting or spline fitting) could be applied as a post-processing step.

Q29: Why does your `find_nearest_node` use brute force instead of a KD-Tree?

A: With `max_iter = 500`, the tree has at most 500 nodes. A brute-force $O(n)$ scan per query means at most $500 \times 500 = 250,000$ distance computations, which is negligible. Building and rebuilding a KD-Tree for a dynamically growing tree adds complexity without meaningful performance gain at this scale. In my PRM planner, where I call nearest-neighbor search 200+ times on a static set, I did use a KD-Tree.

Q30: What happens if RRT fails to find a path?

A: After `max_iter` iterations without reaching the goal, `plan()` prints "Path not found." and returns `None`. This can happen if the map is heavily obstructed, the step size is too small, or we are unlucky with random samples. Solutions: increase `max_iter`, add goal biasing, or adjust `step_size`.

Q31: Why doesn't your RRT explicitly add the goal node to the path?

A: When `reached_goal()` returns `True`, `construct_path()` backtracks from the new node that is within `step_size` of the goal, not from the exact goal position. For a more precise implementation, I could add a final node at the exact goal coordinates as a child of the new node. However, being within `step_size` (5 units on a 100×100 map) is sufficient for most practical applications.

4. PRM QUESTIONS

Q32: Explain PRM in your own words. What are its two phases?

A:

Phase 1 — Construction:

1. Add start and goal to the roadmap.
2. Sample `num_samples` (200) random collision-free points.
3. For each node, find its k nearest neighbors ($k=10$) and try to connect them with collision-free edges.

Phase 2 — Query:

1. Run Dijkstra (or A*) on the constructed roadmap graph.
2. Backtrack from goal to start using parent pointers.

The key advantage is that Phase 1 is done once, and Phase 2 can be repeated for different start/goal pairs without rebuilding the roadmap.

Q33: Why is PRM called “multi-query” and RRT “single-query”?

A: PRM builds a roadmap that captures the connectivity of the free space. Once built, it can answer many different start-to-goal queries by simply running a graph search on the existing roadmap. RRT builds a single tree rooted at one specific start position toward one specific goal. For a different query, a new tree must be built from scratch.

Q34: Why do you add start and goal to the roadmap before sampling?

A: If start and goal are not in the roadmap, no path can connect them. They must participate in the neighbor-finding and edge-building process during construction so that they are connected to nearby samples. Without them, the roadmap would be a disconnected set of points with no way to route from start to goal.

Q35: What is rejection sampling and why do you use it?

A: Rejection sampling generates random points uniformly and rejects those that fall in obstacles. I sample (x, y) randomly and check `self.obstacles.map[x, y]`. If it is True (obstacle), I reject and try again, up to 100 attempts. With $\sim 20\text{--}30\%$ of the map as obstacles, the probability of 100 consecutive rejections is $\sim 0.3^{100} \approx 0$, so 100 attempts is more than sufficient.

Q36: Why do you use Dijkstra instead of A* for the PRM query?

A: Both are correct and guarantee optimal paths on the roadmap graph. Dijkstra is simpler (no heuristic needed). With only ~ 200 nodes, the performance difference is negligible (microseconds). A* with Euclidean heuristic would expand fewer nodes but the total runtime is already instant.

Q37: How do `num_samples` and `k_neighbors` affect PRM?**A:**

- **More samples** → denser roadmap → higher chance of connectivity → better path quality → slower construction.
- **More neighbors** → more edges → higher chance of connectivity → slower construction (more collision checks).
- With 200 samples and `k=10`, the roadmap is usually well-connected for this obstacle configuration (see the figure: 202 nodes, 1114 edges).
- Too few samples or too few neighbors can result in a disconnected roadmap where no path exists.

Q38: What happens if the roadmap is disconnected?

A: If start and goal end up in different connected components of the roadmap, Dijkstra will exhaust all nodes reachable from start without finding the goal. The queue empties and `plan()` returns `None` with “PRM: no path found”. Solutions: increase `num_samples`, increase `k_neighbors`, or use bridge sampling near obstacles.

Q39: Why do you use tuples `(x,y)` as dictionary keys instead of `Node` objects?

A: Python objects hash by identity (memory address) by default, not by content. Two different `Node` objects at the same (x, y) would produce different dictionary keys. Using (x, y) tuples ensures that dictionary lookups match by position value, which is the correct behavior for this algorithm. The `lookup` dictionary maps tuples back to `Node` objects when needed.

Q40: You rebuild the `KDTree` on every call to `find_k_nearest`. Why?

A: The roadmap is built incrementally (samples are added one at a time), so the `KDTree` would need updating. Rebuilding each time is the simplest approach. With 200 nodes, building a `KDTree` is $O(n \log n) \approx O(200 \times 8) \approx 1600$ operations per call, called ~ 200 times = $\sim 320,000$ operations total. This is negligible. For a production system, I would build the tree once after all samples are collected.

5. TRAJECTORY GENERATION QUESTIONS

Q41: What is the purpose of path interpolation?

A: The lattice path is coarse — it only contains lattice vertices (e.g., 15 waypoints for a typical path). `path_interpolation()` samples many intermediate points along each edge (10 points per straight segment, ~15 per arc), creating a dense, smooth path with hundreds of points. This dense path is necessary for the trajectory generator to produce a smooth, well-sampled velocity/acceleration profile.

Q42: Explain the trapezoidal velocity profile. Why is it optimal?

A: The trapezoidal profile has three phases:

1. **Acceleration:** Robot accelerates from rest at `max_acceleration` until reaching `max_velocity`.
2. **Cruise:** Robot maintains `max_velocity`.
3. **Deceleration:** Robot decelerates at `max_acceleration` to stop at the goal.

It is **time-optimal** under constant acceleration constraints because at every moment the robot is either accelerating as hard as possible, cruising at maximum speed, or decelerating as hard as possible. No other profile satisfying the same constraints can complete the path faster.

Q43: Explain the kinematic equation $v_f^2 = v_i^2 + 2as$. Where does it come from?

A: From the two basic kinematic equations:

1. $v_f = v_i + at$ (velocity-time)
2. $s = v_i t + \frac{1}{2}at^2$ (position-time)

Solving equation 1 for $t = (v_f - v_i)/a$ and substituting into equation 2:

$$s = v_i \frac{v_f - v_i}{a} + \frac{1}{2}a \left(\frac{v_f - v_i}{a} \right)^2$$

Simplifying yields $v_f^2 = v_i^2 + 2as$. This equation relates velocity to distance without involving time, which is exactly what we need for the velocity profile (we know distances between stations but want to compute velocities).

Q44: How does the forward pass work?

A: Starting from velocity 0 (at rest), for each station i , I compute the maximum velocity achievable if accelerating at `max_acceleration` from station $i - 1$:

$$v_i = \min \left(\sqrt{v_{i-1}^2 + 2 \cdot a_{max} \cdot \Delta s}, v_{max} \right)$$

This creates an “accelerating from rest” velocity envelope. The `min` clamps to `max_velocity` so the robot never exceeds its speed limit.

Q45: How does the backward pass work?

A: Starting from velocity 0 at the last station (must stop at goal), I iterate backwards. For each station i , I compute the maximum velocity that still allows deceleration to station $i + 1$'s velocity:

$$v_i = \min \left(\sqrt{v_{i+1}^2 + 2 \cdot a_{max} \cdot \Delta s}, v_{max} \right)$$

This creates a “decelerating to rest” velocity envelope.

Q46: Why take the minimum of forward and backward passes?

A: The forward pass gives the maximum velocity considering only acceleration from the start. The backward pass gives the maximum velocity considering only deceleration to stop at the end. The **actual feasible velocity** at each point must satisfy *both* constraints simultaneously: it must be reachable by acceleration AND allow enough distance to decelerate. Taking $\min(fwd, bwd)$ gives the envelope that satisfies both, creating the trapezoidal (or triangular) shape.

Q47: What if the path is too short to reach max_velocity?

A: The forward and backward envelopes intersect before reaching `max_velocity`. The result is a **triangular profile**: accelerate to a peak velocity below `max_velocity`, then immediately decelerate. There is no cruise phase. The $\min(fwd, bwd)$ handles this automatically — no special case is needed.

Q48: What is a “station” in the trajectory?

A: A station is the cumulative arc-length distance from the start of the path to a given waypoint. `stations[0] = 0`, and `stations[i] = stations[i-1] + distance(path[i], path[i-1])`. Stations convert the path from a sequence of positions to a 1D distance parameter, which is needed for the velocity profile computation.

Q49: What is the “gear” and how do you determine it?

A: The gear indicates whether the robot moves forward (1) or in reverse (-1). I compute the **dot product** between the displacement vector (d_{row}, d_{col}) and the robot's heading direction (h_{row}, h_{col}) . If positive, the displacement aligns with the heading (forward motion). If negative, it opposes the heading (reverse motion). The heading is computed from the average angle of consecutive path points.

Q50: Why use $-\sin(\theta)$ for the row component of heading?

A: Grid coordinates have row increasing **downward** (like image coordinates). In standard math, y increases upward. A heading of 90 degrees (“up” in math) means the row **decreases**. So the heading in the row direction is $-\sin(\theta)$. The column direction matches the standard convention: $\cos(\theta)$.

Q51: How do you compute velocity and angular velocity?**A:** Using **backward finite differences**:

$$v_i = \frac{\sqrt{(x_i - x_{i-1})^2 + (y_i - y_{i-1})^2}}{dt}$$

$$\omega_i = \frac{\theta_i - \theta_{i-1}}{dt}$$

For the first point ($i = 0$), I copy from $i = 1$ since there is no $i = -1$. This is a standard boundary condition for numerical differentiation.

Q52: Why not use a centered difference for velocity?

A: A centered difference $(x_{i+1} - x_{i-1})/(2dt)$ is second-order accurate (vs first-order for backward difference), but it requires special handling at *both* boundaries ($i = 0$ and $i = N - 1$). For a first implementation, the backward difference is simpler, sufficient for the assignment, and only needs one boundary fix (at $i = 0$).

Q53: What does `to_continuous_angle` do? Why is it necessary?

A: The lattice angles are in degrees (0, 90, 180, 270). When interpolating, a transition from 350° to 10° has a raw difference of -340° , but the actual angular change is $+20^\circ$. Without unwrapping, the interpolator would make the robot spin 340° the wrong way. `to_continuous_angle` converts to radians, computes the shortest angular difference (normalized to $[-\pi, \pi]$), and accumulates smoothly. The result may exceed $[-\pi, \pi]$ (e.g., 3π for multiple turns), but that is correct for smooth interpolation.

Q54: What does `normalize_angle` do?

A: It wraps any angle to the range $[-\pi, \pi]$ by repeatedly subtracting or adding 2π . This ensures that angular differences represent the **shortest rotation** (e.g., $350^\circ \rightarrow 10^\circ$ is $+20^\circ$, not -340°). For typical angles, the while loop executes 0–1 times.

Q55: Why convert degrees to radians?

A: The lattice uses degrees for convenience (0, 90, 180, 270 are intuitive). But physics formulas use radians: sin, cos, angular velocity ($\omega = d\theta/dt$), and all standard mathematical operations expect radians. The conversion is done in `to_continuous_angle()` before any calculations.

Q56: How is the time profile computed from the velocity profile?**A:** Using $\text{time} = \text{distance}/\text{velocity}$. For each segment:

$$dt_i = \frac{s_i - s_{i-1}}{v_i}$$

The time at each station is the cumulative sum: $t_i = t_{i-1} + dt_i$. This converts from the spatial domain (distance along the path) to the temporal domain, which is needed for uniform time-step interpolation of the trajectory.

Q57: What is the `interpolate_1d` function?

A: It performs 1D linear interpolation. Given known (x, y) pairs and target x values, it finds the corresponding y values by linear interpolation between the two nearest known points. It uses `np.searchsorted` for efficient binary search of the correct interval. This is used to resample x, y, and theta from the time profile onto uniform time steps.

6. UTILITIES / PRE-BUILT CODE QUESTIONS

Q58: How does the Graph class work?

A: It stores:

- `_vert_list`: A list of all vertices as tuples (row, col, angle).
- `_edge_dict`: A dictionary mapping (v1, v2) tuples to edge weights.
- `_adjacency_matrix`: An $N \times N$ numpy array where entry $[j, i]$ equals the weight of edge from vertex i to vertex j (note the transposed indexing convention).

`set_adjacency_matrix()` builds the matrix from the edge dictionary. It is called after graph construction and after obstacle invalidation.

Q59: How does ObstaclesGrid work?

A: It wraps a boolean numpy array (`self.map`) where `True` = obstacle. Key methods:

- `is_edge_valid()`: Determines if an edge is collision-free by sampling points along it. For straight edges (`edge_val == 1`), it samples along a line. For arcs (`edge_val > 1`), it uses precomputed arc primitives.
- `is_point_valid()`: Checks a single point — returns `False` if it is out of bounds or in an obstacle cell.

Q60: How does `get_neighbor` work?

A: Given a vertex u , it finds its row index in `graph_vert_list`, then checks the corresponding row of the adjacency matrix. Any entry that is finite (`< np.inf`) and positive (`> 0`) represents a valid neighbor. It returns a list of all such neighboring vertices. The condition filters out: no edge (0), blocked edges (∞), and negative values (shouldn't exist but safe to exclude).

Q61: What are arc primitives?

A: Precomputed (x, y) sample points along quarter-circle arcs for each possible angle transition (e.g., $0 \rightarrow 90$, $90 \rightarrow 180$). They are generated in `generate_lattice()` (lines 139-183) using parametric equations: $x = \cos(t) \cdot r$ and $y = \sin(t) \cdot r$ for $t \in [0, \pi/2]$. They are used for two purposes: (1) collision checking of arc edges, (2) path interpolation along curved segments.

7. COMPARISON / CONCEPTUAL QUESTIONS

Q62: Compare the three planners in terms of optimality, completeness, and use cases.

A:

	A* Lattice	RRT	PRM
Optimality	Yes (admissible h)	No	No
Completeness	Complete	Prob. complete	Prob. complete
Space	Discrete	Continuous	Continuous
Multi-query	Yes	No	Yes
Orientation	Yes (4 headings)	No	No
Best for	Structured, small grids	Single query, high dim	Repeated queries, static env

Q63: Why does A* give an optimal path but RRT and PRM don't?

A: A* systematically explores all possible paths ordered by cost, guaranteeing that the first path to the goal is optimal (with admissible heuristic). RRT's path depends on random samples and tree structure — it finds *a* path, not the best. PRM's optimality is limited by the roadmap: it finds the shortest path *on the roadmap*, but the roadmap itself is a random approximation of the free space, so the globally optimal path may not be representable.

Q64: When would you choose RRT over A*?

A: When the configuration space is high-dimensional (e.g., a robot arm with 6+ joints), continuous, or has complex kinematic constraints. A* requires discretizing the space into a graph, which becomes intractable in high dimensions (curse of dimensionality). RRT works directly in continuous space and scales well to high dimensions.

Q65: When would you choose PRM over RRT?

A: When you need to solve many different start-to-goal queries in the same static environment. PRM's roadmap is built once (expensive) but each query is fast (just a graph search). With RRT, each new query requires building a new tree from scratch. If the environment changes frequently, RRT is better because PRM's roadmap would need rebuilding.

Q66: Could you combine RRT with the trajectory generator?

A: Yes, but with caveats. RRT produces a 2D path without orientation information, so I would need to assign orientations to each waypoint (e.g., based on the direction of travel). The path would also need smoothing (RRT paths are jagged) before feeding it to the trajectory generator. The trajectory generator expects (x, y, θ) tuples, so I would need to compute θ from consecutive waypoints using $\text{atan2}(dy, dx)$.

Q67: What are the limitations of your implementation?**A:**

- **A***: Only 4 discrete orientations. Limited to grid-based environments. Doesn't scale to high-dimensional spaces.
- **RRT**: No goal biasing (slower convergence). No path smoothing. Not optimal. Finds a different path each run.
- **PRM**: KDTree rebuilt on every query. Not ideal for dynamic environments. Graph search uses Dijkstra (could use A* for speed).
- **Trajectory**: Assumes constant max acceleration/deceleration. No curvature constraints. The gear transition handling is simplified.

Q68: How would you improve each planner?**A:**

- **A***: Add more orientations (8 or 16), use motion primitives for non-holonomic constraints, implement Jump Point Search for faster grid search.
- **RRT**: Add goal biasing (5-10%), implement RRT* for optimality, add path smoothing (shortcutting or splines), use KDTree for nearest-neighbor.
- **PRM**: Build KDTree once after sampling, use A* with heuristic for faster queries, implement lazy PRM (defer collision checking).
- **Trajectory**: Add curvature constraints, implement S-curve (jerk-limited) profiles for smoother motion, handle reverse segments with explicit stop-and-reverse maneuvers.

8. EDGE CASES / TRICKY QUESTIONS

Q69: What if start equals goal?

A: In A*, the start is pushed to the open set, then immediately popped. Since `curr == g` is true, `traverse_path(s, g, parent_node)` is called. Since $s = g$, the while loop `while curr != s` doesn't execute, and the path is just `[s]`. The trajectory generator would produce a single-point trajectory with zero velocity. In RRT, `reached_goal(start)` would be true immediately. In PRM, Dijkstra would pop the start and find it equals the goal.

Q70: What if start or goal is inside an obstacle?

A: For A*, the start/goal are lattice vertices. If all edges from/to that vertex are blocked (crossing obstacles), A* will explore reachable nodes without finding the goal and return None. For RRT/PRM, if the start/goal coordinates are in an obstacle cell, the collision check would flag any edge to/from them as colliding, effectively isolating them. The planners would fail to find a path.

Q71: What if two nodes have the same f-cost in A*?

A: Python's `PriorityQueue` breaks ties by comparing the second element of the tuple, which is the vertex (a tuple of integers). This gives a deterministic but arbitrary tie-breaking order. In theory, tie-breaking can affect which optimal path is found (there may be multiple). A common improvement is to break ties by higher *g*-cost (prefer nodes closer to the goal), which is done with `(f, -g, node)` as the priority tuple.

Q72: What if the velocity profile has a division by zero?

A: This could happen at the endpoints where both forward and backward envelopes give velocity 0 (start from rest, end at rest). The time calculation `ds / profile[i]` would divide by zero. My code handles this with the endpoint fix (lines 150-154): if `profile[0] < 1e-6`, it is set to `profile[1] * 0.5`, and similarly for the last point. Additionally, zero-distance segments (`ds < 1e-9`) are handled by copying the previous velocity.

Q73: Is your collision checking resolution sufficient? Could a thin obstacle be missed?

A: My collision checks sample one point per unit distance (RRT) or one point per `step_size` (PRM). A 1-pixel-thin diagonal obstacle could theoretically be missed if no sample lands exactly on it. However, the obstacles in this assignment are all rectangles at least 3 pixels wide, so the sampling density is adequate. For a production system, I would use Bresenham's line algorithm for pixel-perfect collision checking.

Q74: Why does the RRT find a different path each time?

A: RRT depends on random sampling (`np.random.randint`). Different random seeds produce different sample sequences, different tree structures, and different paths. This is a fundamental property of sampling-based planners. If reproducibility is needed, I could set `np.random.seed()` to a fixed value.

Q75: What would happen if you set `max_velocity` very high (e.g., 1000)?

A: The forward pass would quickly reach `max_velocity` and the backward pass would also. The cruise phase would dominate the profile, with very short acceleration/deceleration phases. The total time would be approximately $\text{total_distance}/v_{max}$. In practice, a very high velocity would be unrealistic and violate the robot's physical capabilities.

Q76: What is the difference between `s_3d` and `s_2d` in `main.py`?

A: `s_3d = (1, 8, 90)` includes orientation (row, col, angle) and is used for the lattice planner, which needs heading information. `s_2d = (1, 8, 90)` is the same values but is used for RRT and PRM, which only use the first two components (row, col) and ignore the angle. Both have the same start position; the distinction is about which planners need orientation.

Q77: How does the trajectory generator handle gear changes (forward to reverse)?

A: When a gear change is detected (`gears[i+1] != gears[i]`), the path is split into segments. Each segment gets its own velocity profile via `generate_optimal_time_profile_segment()`, which starts and ends at zero velocity. This means the robot stops, changes direction, and accelerates again. The time profiles are chained by setting each segment's `start_time` to the previous segment's end time.

9. THEORY / DEEPER UNDERSTANDING

Q78: What is the “curse of dimensionality” and how does it relate to path planning?

A: As the dimension of the configuration space increases, the number of cells in a grid-based discretization grows exponentially. A 2D grid with 100 cells per dimension has $100^2 = 10,000$ cells. A 6D space (robot arm) would have $100^6 = 10^{12}$ cells — impossible to store or search. This is why sampling-based planners (RRT, PRM) are preferred for high-dimensional spaces: they don't discretize the space and their complexity grows moderately with dimension.

Q79: What is a configuration space?

A: The space of all possible configurations (states) of the robot. For a 2D point robot, it's (x, y) — 2D. For a robot with orientation, it's (x, y, θ) — 3D. For a robot arm with n joints, it's n -dimensional. Obstacles in the physical world map to “C-space obstacles” in configuration space. Path planning happens in configuration space.

Q80: What is a non-holonomic constraint? Does your robot have one?

A: A non-holonomic constraint restricts the robot's instantaneous motion without restricting its reachable positions. A car-like robot can't move sideways (it must follow its heading), but it can reach any position/orientation by a sequence of maneuvers. The lattice planner captures non-holonomic behavior: the robot can only move in its current heading direction, and turns require arc edges. RRT and PRM in this implementation don't model non-holonomic constraints — they allow movement in any direction.

Q81: What is the difference between resolution-complete and probabilistically complete?

A: A **resolution-complete** planner (like A* on a grid) will find a path if one exists at the given discretization resolution. If the grid is too coarse, it might miss narrow passages. A **probabilistically complete** planner (like RRT, PRM) will find a path with probability approaching 1 as the number of samples approaches infinity. Neither guarantees finding a path in finite time if the problem is degenerate.

Q82: What makes a velocity profile “time-optimal”?

A: A time-optimal profile minimizes the total time to traverse the path while respecting all constraints (max velocity, max acceleration/deceleration). At every instant, the robot is either:

- Accelerating as hard as possible (forward pass limit), or
- Cruising at max velocity, or
- Decelerating as hard as possible (backward pass limit)

No other profile satisfying the same constraints can complete the trajectory faster. This is a classic result from optimal control theory (bang-bang control).

Q83: What is bang-bang control and how does it relate to your velocity profile?

A: Bang-bang control is an optimal control strategy where the control input (acceleration) is always at its maximum or minimum value — never in between. My trapezoidal profile is a form of bang-bang control: the robot either applies full acceleration ($+a_{max}$), zero acceleration (cruise), or full deceleration ($-a_{max}$). This is time-optimal for the minimum-time point-to-point motion problem under acceleration constraints.

Q84: What would an S-curve velocity profile look like? Why might you prefer it?

A: An S-curve profile limits **jerk** (rate of change of acceleration) in addition to acceleration and velocity. Instead of instantaneous jumps between acceleration phases, the acceleration ramps up and down smoothly. The velocity profile has seven segments: jerk-up, constant-accel, jerk-down, cruise, jerk-down, constant-decel, jerk-up. S-curves produce smoother motion with less vibration and mechanical stress, which is preferred in industrial robotics.